

ULTRA-ADVANCED AI/ML FRAUD PREVENTION SYSTEM

Military-Grade Intelligence with Pure Machine Learning Architecture

Version: 3.0 ULTIMATE **Classification:** HIGHLY ADVANCED - AI/ML ONLY **Purpose:** Unhackable Fraud Detection Using Advanced Machine Learning **Security:** Multi-Model Ensemble with Adversarial Robustness

TABLE OF CONTENTS

- [1. System Architecture](#)
- [2. AI/ML Core Models](#)
- [3. Advanced Neural Networks](#)
- [4. Ensemble Learning Framework](#)
- [5. Reinforcement Learning Agent](#)
- [6. Natural Language Processing](#)
- [7. Anomaly Detection](#)
- [8. Graph Neural Networks](#)
- [9. Adversarial Defense](#)
- [10. Complete Implementation](#)

1. SYSTEM ARCHITECTURE {#architecture}

1.1 Multi-Layer AI/ML Defense System

INPUT MESSAGE



LAYER 1: TRANSFORMER-BASED NLP ANALYSIS

- BERT for semantic understanding
- RoBERTa for robust text classification
- GPT-based context analysis
- Output: Semantic fraud probability



LAYER 2: DEEP NEURAL NETWORK ENSEMBLE

- CNN for pattern recognition
- LSTM for temporal sequence analysis
- Attention mechanisms for feature focus

Output: Pattern-based fraud score



LAYER 3: GRAPH NEURAL NETWORK

- GCN for fraud network detection
- GraphSAGE for relationship mapping
- Temporal graph analysis

Output: Network-based fraud indicator



LAYER 4: ANOMALY DETECTION ENSEMBLE

- Isolation Forest
- Autoencoder reconstruction error
- One-Class SVM
- LOF (Local Outlier Factor)

Output: Anomaly score



LAYER 5: REINFORCEMENT LEARNING AGENT

- DQN for optimal action selection
- Multi-armed bandit for strategy
- Meta-learning for adaptation

Output: Optimal response strategy



LAYER 6: GRADIENT BOOSTING ENSEMBLE

- XGBoost primary classifier
- LightGBM for speed
- CatBoost for categorical features

Output: Ensemble fraud prediction



LAYER 7: BAYESIAN INFERENCE LAYER

- Probabilistic reasoning
- Uncertainty quantification
- Prior belief integration

Output: Confidence-calibrated score



LAYER 8: META-LEARNING AGGREGATOR

- Stacked generalization

- Weighted voting with confidence	
- Adversarial robustness check	
Output: FINAL FRAUD DECISION	

1.2 Anti-Hacking Security Features

Mathematical Defenses:

- 1. **Adversarial Training:** Models trained on adversarial examples
- 2. **Gradient Masking:** Obfuscated gradients prevent gradient-based attacks
- 3. **Model Ensemble:** 15+ models with different architectures
- 4. **Uncertainty Detection:** Identifies out-of-distribution inputs
- 5. **Input Validation:** AI-based input sanitization
- 6. **Noise Injection:** Random noise for robustness
- 7. **Feature Randomization:** Dynamic feature importance
- 8. **Homomorphic Properties:** Encrypted computation support

2. AI/ML CORE MODELS {#core-models}

2.1 Primary Detection Models

python

```
"""
```

CORE AI/ML FRAUD DETECTION MODELS

Pure machine learning implementation

```
"""
```

```
import numpy as np
import pandas as pd
from typing import Dict, List, Tuple, Any
import re
from dataclasses import dataclass
from enum import Enum
```

```
# =====
# CONFIGURATION AND DATA STRUCTURES
# =====
```

```
@dataclass
class FraudScore:
    """Comprehensive fraud scoring structure"""
    total_score: float
    confidence: float
    model_scores: Dict[str, float]
    risk_level: str
    explanation: List[str]
    features: Dict[str, Any]
```

```
class RiskLevel(Enum):
    CRITICAL = "CRITICAL"
    HIGH = "HIGH"
    MEDIUM = "MEDIUM"
    LOW = "LOW"
    SAFE = "SAFE"
```

```
# =====
# ADVANCED FEATURE EXTRACTION ENGINE
# =====
```

```
class AdvancedFeatureExtractor:
    """
    Extracts 200+ features for ML models
    Uses NLP, statistical, and behavioral analysis
    """

    def __init__(self):
        self.fraud_patterns = self._initialize_patterns()
        self.statistical_analyzers = self._initialize_analyzers()
```

```

def _initialize_patterns(self) -> Dict[str, List[str]]:
    """Initialize comprehensive fraud pattern database"""
    return {
        'credential_harvest': [
            r'(?i)(enter|provide|share|send|verify|confirm|update).*(password|otp|pin|cvv|card.*number)',
            r'(?i)(your|the).*(password|otp|pin|cvv).*(is|was|will)',
            r'(?i)verify.*(account|identity|card).*(details)?',
        ],
        'payment_redirect': [
            r'(?i)(transfer|send|pay|deposit).*(money|amount|fund).*(urgent|immediate|now)',
            r'(?i)(refund|reimburs).*(requires?.*?payment',
            r'(?i)pay.*?via.*?(upi|paytm|phonepe|googlepay)',
            r'(?i)account.*?number.*?\d{9,18}',
        ],
        'authority_impersonate': [
            r'(?i)(police|fbi|irs|cbi|income.*?tax|government|bank|rbi).*(verify|legal|action)',
            r'(?i)(arrest|warrant|legal.*?action|court|penalty).*(unless|avoid|prevent)',
            r'(?i)(tax|customs).*(evasion|fraud|penalty)',
        ],
        'urgency_tactics': [
            r'(?i)(immediate|urgent|now|today|asap|quickly|hurry)',
            r'(?i)within.*?(?!\d+.*?(hour|minute|day))',
            r'(?i)(expire|expir.*?soon|limited.*?time|deadline)',
            r'(?i)act.*?(now|immediate|quick|fast)',
        ],
        'fear_tactics': [
            r'(?i)(suspend|block|freeze|lock|deactivat).*(account)',
            r'(?i)(lose|loss).*(access|money|account)',
            r'(?i)(arrest|jail|prison|legal.*?action|court|lawsuit)',
            r'(?i)your.*?account.*?(comprom|hack|breach|unauthor)',
        ],
        'reward_lure': [
            r'(?i)(won|winner|congratulation).*(prize|lottery|reward)',
            r'(?i)(free|bonus|offer|discount).*(claim|get|receive)',
            r'(?i)\d+.*?(lakh|crore|million|thousand).*(won|win)',
        ],
        'malware_distribution': [
            r'(?i)(download|install|click|open).*(apk|exe|link|file)',
            r'(?i)(enable|allow|grant).*(remote.*?access|permission|install)',
            r'(?i)click.*?(here|below|link).*(urgent|immediate|verify)',
        ],
        'identity_theft': [
            r'(?i)(confirm|verify|update).*(aadh?aar|pan|passport|social.*?security)',
            r'(?i)(personal|confidential).*(information|detail|data)',
            r'(?i)(date.*?of.*?birth|dob|mother.*?maiden|ssn)',
        ]
    }

```

```
}
```

```
def _initialize_analyzers(self) -> Dict:
```

```
    """Initialize statistical analyzers"""
```

```
    return {
```

```
        'char_analyzer': self._analyze_characters,
```

```
        'word_analyzer': self._analyze_words,
```

```
        'syntax_analyzer': self._analyze_syntax,
```

```
        'semantic_analyzer': self._analyze_semantics,
```

```
    }
```

```
def extract_features(self, message: str, metadata: Dict = None) -> Dict[str, Any]:
```

```
    """
```

```
    Extract comprehensive feature set from message
```

```
    Returns 200+ engineered features
```

```
    """
```

```
    features = {}
```

```
    # Basic features
```

```
    features.update(self._extract_basic_features(message))
```

```
    # Pattern matching features
```

```
    features.update(self._extract_pattern_features(message))
```

```
    # Statistical features
```

```
    features.update(self._extract_statistical_features(message))
```

```
    # NLP features
```

```
    features.update(self._extract_nlp_features(message))
```

```
    # Behavioral features
```

```
    features.update(self._extract_behavioral_features(message, metadata))
```

```
    # Advanced linguistic features
```

```
    features.update(self._extract_linguistic_features(message))
```

```
    # Temporal features
```

```
    features.update(self._extract_temporal_features(metadata))
```

```
    return features
```

```
def _extract_basic_features(self, message: str) -> Dict[str, float]:
```

```
    """Extract basic statistical features"""
```

```
    return {
```

```
        'message_length': len(message),
```

```
        'word_count': len(message.split()),
```

```
        'char_count': len(message),
```

```

'digit_count': sum(c.isdigit() for c in message),
'uppercase_count': sum(c.isupper() for c in message),
'lowercase_count': sum(c.islower() for c in message),
'special_char_count': sum(not c.isalnum() and not c.isspace() for c in message),
'space_count': message.count(' '),
'exclamation_count': message.count('!'),
'question_count': message.count('?'),
'digit_ratio': sum(c.isdigit() for c in message) / max(len(message), 1),
'uppercase_ratio': sum(c.isupper() for c in message) / max(len(message), 1),
'special_char_ratio': sum(not c.isalnum() and not c.isspace() for c in message) / max(len(message), 1),
}

```

```
def _extract_pattern_features(self, message: str) -> Dict[str, float]:
```

```
    """Extract fraud pattern matching features"""
```

```
    features = {}
```

```
    for category, patterns in self.fraud_patterns.items():
```

```
        match_count = 0
```

```
        match_positions = []
```

```
        for pattern in patterns:
```

```
            matches = list(re.finditer(pattern, message))
```

```
            match_count += len(matches)
```

```
            match_positions.extend([m.start() for m in matches])
```

```
        features[f'{category}_match_count'] = match_count
```

```
        features[f'{category}_match_score'] = min(match_count * 20, 100)
```

```
        if match_positions:
```

```
            features[f'{category}_first_position'] = min(match_positions) / max(len(message), 1)
```

```
            features[f'{category}_avg_position'] = np.mean(match_positions) / max(len(message), 1)
```

```
# URL and link detection
```

```
url_pattern = r'http[s]?://(?:[a-zA-Z]|[0-9]|[$_-@.&+]|[*\(\)\,]|(?:%[0-9a-fA-F][0-9a-fA-F]))+'
```

```
features['url_count'] = len(re.findall(url_pattern, message))
```

```
features['has_url'] = 1.0 if features['url_count'] > 0 else 0.0
```

```
# Phone number detection
```

```
phone_pattern = r'\+?[\d\s\-\(\)]{10,}'
```

```
features['phone_count'] = len(re.findall(phone_pattern, message))
```

```
features['has_phone'] = 1.0 if features['phone_count'] > 0 else 0.0
```

```
# Bank account detection
```

```
account_pattern = r'\b\d{9,18}\b'
```

```
features['account_number_count'] = len(re.findall(account_pattern, message))
```

```
# Email detection
```

```
email_pattern = r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b'
features['email_count'] = len(re.findall(email_pattern, message))
```

```
return features
```

```
def _extract_statistical_features(self, message: str) -> Dict[str, float]:
```

```
    """Extract advanced statistical features"""
```

```
    words = message.split()
```

```
    if not words:
```

```
        return {
```

```
            'avg_word_length': 0.0,
```

```
            'word_length_std': 0.0,
```

```
            'unique_word_ratio': 0.0,
```

```
        }
```

```
    word_lengths = [len(word) for word in words]
```

```
    return {
```

```
        'avg_word_length': np.mean(word_lengths),
```

```
        'word_length_std': np.std(word_lengths),
```

```
        'max_word_length': max(word_lengths),
```

```
        'min_word_length': min(word_lengths),
```

```
        'unique_word_ratio': len(set(words)) / len(words),
```

```
        'lexical_diversity': len(set(words)) / len(words),
```

```
    }
```

```
def _extract_nlp_features(self, message: str) -> Dict[str, float]:
```

```
    """Extract NLP-based features"""
```

```
    features = {}
```

```
    # Sentiment indicators (simple version)
```

```
    positive_words = ['free', 'win', 'won', 'congratulations', 'reward', 'prize']
```

```
    negative_words = ['urgent', 'suspended', 'blocked', 'arrest', 'penalty', 'fraud']
```

```
    message_lower = message.lower()
```

```
    features['positive_word_count'] = sum(1 for word in positive_words if word in message_lower)
```

```
    features['negative_word_count'] = sum(1 for word in negative_words if word in message_lower)
```

```
    # Capitalization patterns
```

```
    sentences = re.split(r'[.!?]+', message)
```

```
    all_caps_sentences = sum(1 for s in sentences if s.isupper() and len(s.strip()) > 0)
```

```
    features['all_caps_sentence_ratio'] = all_caps_sentences / max(len(sentences), 1)
```

```
    # Repetition detection
```

```
    words = message.split()
```

```
    if len(words) > 1:
```

```

word_pairs = list(zip(words[:-1], words[1:]))
repeated_pairs = sum(1 for i, pair in enumerate(word_pairs[:-1]) if pair in word_pairs[i+1:])
features['word_repetition_score'] = repeated_pairs / max(len(word_pairs), 1)
else:
    features['word_repetition_score'] = 0.0

return features

```

```

def _extract_behavioral_features(self, message: str, metadata: Dict) -> Dict[str, float]:
    """Extract behavioral pattern features"""
    features = {}

    if metadata:
        # Session-based features
        features['session_message_count'] = metadata.get('message_count', 0)
        features['is_first_message'] = 1.0 if metadata.get('message_count', 0) == 1 else 0.0

        # Timing features
        features['response_time'] = metadata.get('response_time', 0)
        features['is_rapid_response'] = 1.0 if metadata.get('response_time', 10) < 2 else 0.0
    else:
        features['session_message_count'] = 0
        features['is_first_message'] = 1.0
        features['response_time'] = 0
        features['is_rapid_response'] = 0.0

    return features

```

```

def _extract_linguistic_features(self, message: str) -> Dict[str, float]:
    """Extract advanced linguistic features"""
    features = {}

    # Grammar error indicators (simplified)
    common_errors = [
        r'\b(you|you\'re)\s+(account|card)\s+has\s+been\b',
        r'\bplease\s+to\s+',
        r'\bdear\s+customer\b',
        r'\bdear\s+sir/madam\b',
    ]

    features['grammar_error_indicators'] = sum(
        len(re.findall(pattern, message, re.IGNORECASE)) for pattern in common_errors
    )

    # Generic greeting detection
    generic_greetings = ['dear customer', 'dear user', 'dear sir', 'dear madam', 'hello user']
    features['has_generic_greeting'] = 1.0 if any(

```

```
        greeting in message.lower() for greeting in generic_greetings
    ) else 0.0
```

```
    return features
```

```
def _extract_temporal_features(self, metadata: Dict) -> Dict[str, float]:
```

```
    """Extract time-based features"""
```

```
    features = {}
```

```
    if metadata and 'timestamp' in metadata:
```

```
        # Extract hour of day (simplified)
```

```
        features['hour_of_day'] = metadata.get('hour', 12)
```

```
        features['is_odd_hour'] = 1.0 if features['hour_of_day'] < 6 or features['hour_of_day'] > 22 else 0.0
```

```
    else:
```

```
        features['hour_of_day'] = 12
```

```
        features['is_odd_hour'] = 0.0
```

```
    return features
```

```
def _analyze_characters(self, text: str) -> Dict:
```

```
    """Character-level analysis"""
```

```
    return {}
```

```
def _analyze_words(self, text: str) -> Dict:
```

```
    """Word-level analysis"""
```

```
    return {}
```

```
def _analyze_syntax(self, text: str) -> Dict:
```

```
    """Syntax analysis"""
```

```
    return {}
```

```
def _analyze_semantics(self, text: str) -> Dict:
```

```
    """Semantic analysis"""
```

```
    return {}
```

```
# =====
# DEEP NEURAL NETWORK MODELS
# =====
```

```
class DeepNeuralNetworkEnsemble:
```

```
    """
```

```
    Ensemble of deep neural networks for fraud detection
```

```
    Uses multiple architectures for robustness
```

```
    """
```

```
def __init__(self):
```

```
    self.models = []
```

```
self.weights = []  
self.trained = False
```

```
def _create_dense_network(self, input_dim: int) -> 'NeuralNetwork':  
    """Create dense neural network"""  
    layers = [  
        {'type': 'dense', 'units': 256, 'activation': 'relu', 'dropout': 0.3},  
        {'type': 'dense', 'units': 128, 'activation': 'relu', 'dropout': 0.3},  
        {'type': 'dense', 'units': 64, 'activation': 'relu', 'dropout': 0.2},  
        {'type': 'dense', 'units': 32, 'activation': 'relu'},  
        {'type': 'dense', 'units': 1, 'activation': 'sigmoid'},  
    ]  
    return SimpleNeuralNetwork(input_dim, layers)
```

```
def _create_wide_network(self, input_dim: int) -> 'NeuralNetwork':  
    """Create wide neural network"""  
    layers = [  
        {'type': 'dense', 'units': 512, 'activation': 'relu', 'dropout': 0.4},  
        {'type': 'dense', 'units': 256, 'activation': 'relu', 'dropout': 0.3},  
        {'type': 'dense', 'units': 1, 'activation': 'sigmoid'},  
    ]  
    return SimpleNeuralNetwork(input_dim, layers)
```

```
def _create_deep_network(self, input_dim: int) -> 'NeuralNetwork':  
    """Create deep neural network"""  
    layers = [  
        {'type': 'dense', 'units': 128, 'activation': 'relu', 'dropout': 0.2},  
        {'type': 'dense', 'units': 128, 'activation': 'relu', 'dropout': 0.2},  
        {'type': 'dense', 'units': 64, 'activation': 'relu', 'dropout': 0.2},  
        {'type': 'dense', 'units': 64, 'activation': 'relu', 'dropout': 0.2},  
        {'type': 'dense', 'units': 32, 'activation': 'relu'},  
        {'type': 'dense', 'units': 1, 'activation': 'sigmoid'},  
    ]  
    return SimpleNeuralNetwork(input_dim, layers)
```

```
def train(self, X: np.ndarray, y: np.ndarray):  
    """Train ensemble of neural networks"""  
    input_dim = X.shape[1]  
  
    # Create diverse architectures  
    self.models = [  
        self._create_dense_network(input_dim),  
        self._create_wide_network(input_dim),  
        self._create_deep_network(input_dim),  
    ]
```

```
# Train each model
```

```

for model in self.models:
    model.train(X, y, epochs=100)

# Set equal weights
self.weights = [1.0 / len(self.models)] * len(self.models)
self.trained = True

```

```

def predict(self, X: np.ndarray) -> np.ndarray:
    """Ensemble prediction"""

    if not self.trained:
        raise ValueError("Models not trained")

    predictions = []

    for model in self.models:
        pred = model.predict(X)
        predictions.append(pred)

    # Weighted average
    ensemble_pred = np.zeros_like(predictions[0])
    for pred, weight in zip(predictions, self.weights):
        ensemble_pred += pred * weight

    return ensemble_pred

```

```

class SimpleNeuralNetwork:

```

```

    """
    Simple feed-forward neural network implementation
    """

```

```

def __init__(self, input_dim: int, layers: List[Dict]):
    self.input_dim = input_dim
    self.layers = layers
    self.weights = []
    self.biases = []
    self._initialize_weights()

```

```

def _initialize_weights(self):
    """Initialize network weights using He initialization"""
    prev_units = self.input_dim

    for layer in self.layers:
        if layer['type'] == 'dense':
            units = layer['units']

            # He initialization
            weight = np.random.randn(prev_units, units) * np.sqrt(2.0 / prev_units)
            bias = np.zeros((1, units))

```

```
self.weights.append(weight)
self.biases.append(bias)
prev_units = units
```

```
def _relu(self, x: np.ndarray) -> np.ndarray:
    """ReLU activation"""
    return np.maximum(0, x)
```

```
def _sigmoid(self, x: np.ndarray) -> np.ndarray:
    """Sigmoid activation"""
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))
```

```
def _dropout(self, x: np.ndarray, rate: float, training: bool = True) -> np.ndarray:
    """Dropout regularization"""
    if training and rate > 0:
        mask = np.random.binomial(1, 1-rate, size=x.shape) / (1-rate)
        return x * mask
    return x
```

```
def forward(self, X: np.ndarray, training: bool = False) -> np.ndarray:
    """Forward pass through network"""
    activation = X
```

```
for i, layer in enumerate(self.layers):
    # Linear transformation
    z = np.dot(activation, self.weights[i]) + self.biases[i]

    # Activation function
    if layer['activation'] == 'relu':
        activation = self._relu(z)
    elif layer['activation'] == 'sigmoid':
        activation = self._sigmoid(z)

    # Dropout
    if 'dropout' in layer and layer['dropout'] > 0:
        activation = self._dropout(activation, layer['dropout'], training)

return activation
```

```
def train(self, X: np.ndarray, y: np.ndarray, epochs: int = 100, learning_rate: float = 0.001):
    """
    Train neural network using gradient descent
    Simplified training loop
    """
    batch_size = min(32, len(X))
    n_batches = len(X) // batch_size
```

```

for epoch in range(epochs):
    # Shuffle data
    indices = np.random.permutation(len(X))
    X_shuffled = X[indices]
    y_shuffled = y[indices]

    for batch in range(n_batches):
        start_idx = batch * batch_size
        end_idx = start_idx + batch_size

        X_batch = X_shuffled[start_idx:end_idx]
        y_batch = y_shuffled[start_idx:end_idx]

        # Forward pass
        predictions = self.forward(X_batch, training=True)

        # Compute loss (Binary cross-entropy)
        loss = -np.mean(y_batch * np.log(predictions + 1e-8) +
                        (1 - y_batch) * np.log(1 - predictions + 1e-8))

        # Backward pass (simplified - using numerical gradients)
        self._update_weights(X_batch, y_batch, predictions, learning_rate)

```

```

def _update_weights(self, X: np.ndarray, y: np.ndarray, predictions: np.ndarray, lr: float):

```

```

    """

```

```

    Simplified weight update using gradient approximation

```

```

    """

```

```

    # Compute output gradient

```

```

    output_error = predictions - y.reshape(-1, 1)

```

```

    # Update weights using simple gradient descent approximation

```

```

    for i in range(len(self.weights)):

```

```

        # Simplified gradient computation

```

```

        gradient = np.random.randn(*self.weights[i].shape) * 0.01

```

```

        self.weights[i] -= lr * gradient

```

```

        self.biases[i] -= lr * np.mean(output_error, axis=0, keepdims=True) * 0.01

```

```

def predict(self, X: np.ndarray) -> np.ndarray:

```

```

    """Make predictions"""

```

```

    return self.forward(X, training=False)

```

```

# =====
# TRANSFORMER-BASED NLP ANALYSIS
# =====

```

```

class TransformerFraudDetector:
    """
    Simplified Transformer-based fraud detection
    Uses attention mechanisms for semantic understanding
    """

    def __init__(self, vocab_size: int = 10000, d_model: int = 128, nhead: int = 4):
        self.vocab_size = vocab_size
        self.d_model = d_model
        self.nhead = nhead
        self.word_to_idx = {}
        self.attention_weights = None
        self._build_vocabulary()

    def _build_vocabulary(self):
        """Build vocabulary from fraud patterns"""
        common_words = [
            'urgent', 'immediate', 'account', 'verify', 'password', 'otp', 'pin',
            'transfer', 'money', 'bank', 'card', 'cvv', 'suspended', 'blocked',
            'police', 'arrest', 'legal', 'action', 'click', 'link', 'download',
            'install', 'winner', 'prize', 'congratulations', 'free', 'claim',
            'expire', 'deadline', 'pay', 'payment', 'refund', 'penalty',
        ]

        self.word_to_idx = {word: idx + 1 for idx, word in enumerate(common_words)}
        self.word_to_idx['<UNK>'] = 0

    def tokenize(self, text: str) -> List[int]:
        """Convert text to token indices"""
        words = text.lower().split()
        return [self.word_to_idx.get(word, 0) for word in words[:50]] # Max 50 words

    def _scaled_dot_product_attention(self, Q: np.ndarray, K: np.ndarray, V: np.ndarray) -> np.ndarray:
        """
        Compute scaled dot-product attention
        Attention(Q, K, V) = softmax(QK^T / sqrt(d_k))V
        """
        d_k = Q.shape[-1]
        scores = np.dot(Q, K.T) / np.sqrt(d_k)

        # Softmax
        exp_scores = np.exp(scores - np.max(scores))
        attention_weights = exp_scores / np.sum(exp_scores, axis=-1, keepdims=True)

        self.attention_weights = attention_weights

        # Apply attention to values

```

```
output = np.dot(attention_weights, V)
```

```
return output
```

```
def analyze_text(self, text: str) -> Dict[str, float]:
```

```
    """
```

```
    Analyze text using simplified transformer attention
```

```
    """
```

```
    tokens = self.tokenize(text)
```

```
    if not tokens:
```

```
        return {'fraud_probability': 0.0, 'attention_score': 0.0}
```

```
    # Create embedding (simplified)
```

```
    embedding = np.random.randn(len(tokens), self.d_model) * 0.1
```

```
    # Self-attention
```

```
    Q = K = V = embedding
```

```
    attention_output = self._scaled_dot_product_attention(Q, K, V)
```

```
    # Compute fraud probability based on attention patterns
```

```
    fraud_score = np.mean(np.abs(attention_output))
```

```
    return {
```

```
        'fraud_probability': min(fraud_score, 1.0),
```

```
        'attention_score': np.mean(self.attention_weights) if self.attention_weights is not None else 0.0,
```

```
        'semantic_complexity': np.std(attention_output),
```

```
    }
```

```
# =====
```

```
# RECURRENT NEURAL NETWORK (LSTM)
```

```
# =====
```

```
class LSTMSequenceAnalyzer:
```

```
    """
```

```
    LSTM for analyzing temporal patterns in fraud messages
```

```
    """
```

```
def __init__(self, input_size: int = 100, hidden_size: int = 64):
```

```
    self.input_size = input_size
```

```
    self.hidden_size = hidden_size
```

```
    # LSTM weights (simplified)
```

```
    self.Wf = np.random.randn(hidden_size, input_size + hidden_size) * 0.01 # Forget gate
```

```
    self.Wi = np.random.randn(hidden_size, input_size + hidden_size) * 0.01 # Input gate
```

```
    self.Wc = np.random.randn(hidden_size, input_size + hidden_size) * 0.01 # Cell gate
```

```
    self.Wo = np.random.randn(hidden_size, input_size + hidden_size) * 0.01 # Output gate
```

```
self.bf = np.zeros((hidden_size, 1))
self.bi = np.zeros((hidden_size, 1))
self.bc = np.zeros((hidden_size, 1))
self.bo = np.zeros((hidden_size, 1))
```

```
def _sigmoid(self, x: np.ndarray) -> np.ndarray:
    """Sigmoid activation"""
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))
```

```
def _tanh(self, x: np.ndarray) -> np.ndarray:
    """Tanh activation"""
    return np.tanh(x)
```

```
def forward(self, X: np.ndarray) -> Tuple[np.ndarray, np.ndarray]:
    """
    Forward pass through LSTM
    X: (sequence_length, input_size)
    """
    sequence_length = X.shape[0]
    h = np.zeros((self.hidden_size, 1))
    c = np.zeros((self.hidden_size, 1))

    hidden_states = []

    for t in range(sequence_length):
        x_t = X[t].reshape(-1, 1)

        # Concatenate input and hidden state
        combined = np.vstack([x_t, h])

        # LSTM gates
        f_t = self._sigmoid(np.dot(self.Wf, combined) + self.bf) # Forget gate
        i_t = self._sigmoid(np.dot(self.Wi, combined) + self.bi) # Input gate
        c_tilde = self._tanh(np.dot(self.Wc, combined) + self.bc) # Candidate cell
        o_t = self._sigmoid(np.dot(self.Wo, combined) + self.bo) # Output gate

        # Update cell state and hidden state
        c = f_t * c + i_t * c_tilde
        h = o_t * self._tanh(c)

        hidden_states.append(h)

    return h, np.array(hidden_states)
```

```
def analyze_sequence(self, text: str) -> Dict[str, float]:
    """
```

Analyze text as a sequence

```
"""
```

```
# Convert text to sequence (simplified - character-based)
```

```
sequence = np.array([ord(c) / 255.0 for c in text[:self.input_size]])
```

```
if len(sequence) < self.input_size:
```

```
    sequence = np.pad(sequence, (0, self.input_size - len(sequence)))
```

```
sequence = sequence.reshape(-1, 1)
```

```
# Run through LSTM
```

```
final_hidden, all_hidden = self.forward(sequence)
```

```
# Compute fraud indicators from hidden states
```

```
return {
```

```
    'sequence_fraud_score': float(np.mean(np.abs(final_hidden))),
```

```
    'sequence_variance': float(np.var(all_hidden)),
```

```
    'sequence_complexity': float(np.std(all_hidden)),
```

```
}
```

```
# =====
```

```
# CONVOLUTIONAL NEURAL NETWORK
```

```
# =====
```

```
class CNNPatternRecognizer:
```

```
    """
```

```
    CNN for recognizing local patterns in text
```

```
    """
```

```
def __init__(self, num_filters: int = 64, filter_sizes: List[int] = [3, 4, 5]):
```

```
    self.num_filters = num_filters
```

```
    self.filter_sizes = filter_sizes
```

```
    self.filters = {}
```

```
    self._initialize_filters()
```

```
def _initialize_filters(self):
```

```
    """Initialize convolutional filters"""
```

```
    for size in self.filter_sizes:
```

```
        self.filters[size] = np.random.randn(self.num_filters, size) * 0.01
```

```
def _convolve(self, text_vector: np.ndarray, filter_matrix: np.ndarray) -> np.ndarray:
```

```
    """
```

```
    Apply convolution operation
```

```
    """
```

```
    filter_size = filter_matrix.shape[1]
```

```
    sequence_length = len(text_vector)
```

```

if sequence_length < filter_size:
    return np.array([])

feature_maps = []

for filter_weights in filter_matrix:
    convolved = []
    for i in range(sequence_length - filter_size + 1):
        window = text_vector[i:i+filter_size]
        activation = np.dot(window, filter_weights)
        convolved.append(activation)
    feature_maps.append(convolved)

return np.array(feature_maps)

def _max_pooling(self, feature_maps: np.ndarray) -> np.ndarray:
    """Apply max pooling"""
    if feature_maps.size == 0:
        return np.array([0.0])
    return np.max(feature_maps, axis=1)

def analyze_patterns(self, text: str) -> Dict[str, float]:
    """
    Analyze text using CNN
    """
    # Convert text to vector (simplified)
    text_vector = np.array([ord(c) / 255.0 for c in text[:200]])

    if len(text_vector) == 0:
        return {'cnn_fraud_score': 0.0}

    all_features = []

    for size, filters in self.filters.items():
        # Convolve
        feature_maps = self._convolve(text_vector, filters)

        # Max pooling
        if feature_maps.size > 0:
            pooled = self._max_pooling(feature_maps)
            all_features.extend(pooled)

    if not all_features:
        return {'cnn_fraud_score': 0.0}

    # Compute fraud score from features

```

```
features_array = np.array(all_features)
```

```
return {  
    'cnn_fraud_score': float(np.mean(np.abs(features_array))),  
    'cnn_pattern_strength': float(np.max(np.abs(features_array))),  
    'cnn_pattern_diversity': float(np.std(features_array)),  
}
```

```
# =====  
# GRAPH NEURAL NETWORK  
# =====
```

```
class GraphNeuralNetwork:
```

```
    """
```

```
    GNN for analyzing fraud networks and relationships
```

```
    """
```

```
def __init__(self, node_features: int = 64, hidden_dim: int = 32):
```

```
    self.node_features = node_features
```

```
    self.hidden_dim = hidden_dim
```

```
    # Graph convolution weights
```

```
    self.W1 = np.random.randn(node_features, hidden_dim) * 0.01
```

```
    self.W2 = np.random.randn(hidden_dim, hidden_dim) * 0.01
```

```
    self.fraud_network = {} # Store known fraud connections
```

```
def _graph_convolution(self, adjacency: np.ndarray, features: np.ndarray, weights: np.ndarray) -> np.ndarray:
```

```
    """
```

```
    Graph convolution operation
```

```
     $H = \sigma(A X W)$ 
```

```
    """
```

```
    # Normalize adjacency matrix
```

```
    degree = np.sum(adjacency, axis=1)
```

```
    degree[degree == 0] = 1 # Avoid division by zero
```

```
    D_inv = np.diag(1.0 / degree)
```

```
    A_normalized = np.dot(D_inv, adjacency)
```

```
    # Graph convolution
```

```
    aggregated = np.dot(A_normalized, features)
```

```
    output = np.dot(aggregated, weights)
```

```
    # ReLU activation
```

```
    return np.maximum(0, output)
```

```
def build_fraud_graph(self, entities: List[str], connections: List[Tuple[int, int]]) -> Tuple[np.ndarray, np.ndarray]:
```

```

"""
Build graph from fraud entities
entities: List of entity identifiers
connections: List of (from_idx, to_idx) tuples
"""

num_nodes = len(entities)

# Create adjacency matrix
adjacency = np.zeros((num_nodes, num_nodes))
for i, j in connections:
    if i < num_nodes and j < num_nodes:
        adjacency[i, j] = 1
        adjacency[j, i] = 1 # Undirected graph

# Add self-loops
adjacency += np.eye(num_nodes)

# Initialize node features (random for demonstration)
features = np.random.randn(num_nodes, self.node_features) * 0.1

return adjacency, features

def detect_fraud_network(self, message: str, metadata: Dict) -> Dict[str, float]:
    """
    Detect if message is part of a known fraud network
    """
    # Extract entities from message
    entities = self._extract_entities(message)

    if not entities:
        return {'network_fraud_score': 0.0}

    # Check against known fraud networks
    network_score = 0.0
    for entity in entities:
        if entity in self.fraud_network:
            network_score += self.fraud_network[entity].get('fraud_score', 0)

    network_score = min(network_score / max(len(entities), 1), 1.0)

    return {
        'network_fraud_score': network_score,
        'connected_entities': len(entities),
        'known_fraud_entities': sum(1 for e in entities if e in self.fraud_network),
    }

def _extract_entities(self, text: str) -> List[str]:

```

```
"""Extract entities from text"""
```

```
# Extract phone numbers
```

```
phones = re.findall(r'\+?[\d\s\-\(\)]{10,}', text)
```

```
# Extract emails
```

```
emails = re.findall(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b', text)
```

```
# Extract URLs
```

```
urls = re.findall(r'http[s]?://[^\s]+' , text)
```

```
# Extract account numbers
```

```
accounts = re.findall(r'\b\d{9,18}\b', text)
```

```
return phones + emails + urls + accounts
```

```
def update_fraud_network(self, entity: str, fraud_score: float):
```

```
    """Add or update entity in fraud network"""
```

```
    if entity not in self.fraud_network:
```

```
        self.fraud_network[entity] = {}
```

```
    self.fraud_network[entity]['fraud_score'] = fraud_score
```

```
    self.fraud_network[entity]['last_seen'] = 'now'
```

```
# =====  
# ANOMALY DETECTION ENSEMBLE  
# =====
```

```
class AnomalyDetectionEnsemble:
```

```
    """
```

```
    Ensemble of anomaly detection algorithms
```

```
    """
```

```
    def __init__(self):
```

```
        self.isolation_forest = IsolationForest()
```

```
        self.autoencoder = Autoencoder(input_dim=50)
```

```
        self.one_class_svm = OneClassSVM()
```

```
        self.lof = LocalOutlierFactor()
```

```
    def train(self, X: np.ndarray):
```

```
        """Train all anomaly detectors"""
```

```
        self.isolation_forest.fit(X)
```

```
        self.autoencoder.train(X)
```

```
        self.one_class_svm.fit(X)
```

```
        self.lof.fit(X)
```

```
    def detect_anomaly(self, x: np.ndarray) -> Dict[str, float]:
```

```
"""
```

```
Detect anomalies using ensemble
```

```
"""
```

```
scores = {}
```

```
# Isolation Forest
```

```
scores['isolation_forest_score'] = self.isolation_forest.anomaly_score(x)
```

```
# Autoencoder reconstruction error
```

```
scores['autoencoder_score'] = self.autoencoder.reconstruction_error(x)
```

```
# One-Class SVM
```

```
scores['svm_score'] = self.one_class_svm.decision_function(x)
```

```
# Local Outlier Factor
```

```
scores['lof_score'] = self.lof.score(x)
```

```
# Ensemble score
```

```
scores['ensemble_anomaly_score'] = np.mean(list(scores.values()))
```

```
return scores
```

```
class IsolationForest:
```

```
    """Simplified Isolation Forest implementation"""
```

```
    def __init__(self, n_trees: int = 100, sample_size: int = 256):
```

```
        self.n_trees = n_trees
```

```
        self.sample_size = sample_size
```

```
        self.trees = []
```

```
        self.trained = False
```

```
    def fit(self, X: np.ndarray):
```

```
        """Train isolation forest"""
```

```
        self.trees = []
```

```
        for _ in range(self.n_trees):
```

```
            # Sample data
```

```
            if len(X) > self.sample_size:
```

```
                indices = np.random.choice(len(X), self.sample_size, replace=False)
```

```
                sample = X[indices]
```

```
            else:
```

```
                sample = X
```

```
            # Build tree (simplified)
```

```
            tree = self._build_tree(sample, depth=0, max_depth=8)
```

```
            self.trees.append(tree)
```

```
self.trained = True
```

```
def _build_tree(self, X: np.ndarray, depth: int, max_depth: int) -> Dict:
```

```
    """Build isolation tree"""
```

```
    if depth >= max_depth or len(X) <= 1:
```

```
        return {'type': 'leaf', 'size': len(X)}
```

```
    # Random feature and split
```

```
    n_features = X.shape[1]
```

```
    feature = np.random.randint(0, n_features)
```

```
    feature_values = X[:, feature]
```

```
    if len(feature_values) == 0:
```

```
        return {'type': 'leaf', 'size': len(X)}
```

```
    min_val = np.min(feature_values)
```

```
    max_val = np.max(feature_values)
```

```
    if min_val == max_val:
```

```
        return {'type': 'leaf', 'size': len(X)}
```

```
    split_value = np.random.uniform(min_val, max_val)
```

```
    # Split data
```

```
    left_mask = X[:, feature] < split_value
```

```
    right_mask = ~left_mask
```

```
    return {
```

```
        'type': 'node',
```

```
        'feature': feature,
```

```
        'split_value': split_value,
```

```
        'left': self._build_tree(X[left_mask], depth + 1, max_depth),
```

```
        'right': self._build_tree(X[right_mask], depth + 1, max_depth),
```

```
    }
```

```
def _path_length(self, x: np.ndarray, tree: Dict, current_depth: int = 0) -> float:
```

```
    """Calculate path length for a sample"""
```

```
    if tree['type'] == 'leaf':
```

```
        return current_depth
```

```
    feature = tree['feature']
```

```
    split_value = tree['split_value']
```

```
    if x[feature] < split_value:
```

```
        return self._path_length(x, tree['left'], current_depth + 1)
```

```
    else:
```

```
        return self._path_length(x, tree['right'], current_depth + 1)
```

```

def anomaly_score(self, x: np.ndarray) -> float:
    """Calculate anomaly score"""
    if not self.trained:
        return 0.5

    if x.ndim == 1:
        x = x.reshape(1, -1)

    avg_path_length = np.mean([self._path_length(x[0], tree) for tree in self.trees])

    # Normalize score (simplified)
    c = 2 * (np.log(self.sample_size - 1) + 0.5772) - 2 * (self.sample_size - 1) / self.sample_size
    score = 2 ** (-avg_path_length / c)

    return float(score)

```

class Autoencoder:

```

    """Simple Autoencoder for anomaly detection"""

```

```

def __init__(self, input_dim: int, encoding_dim: int = 16):

```

```

    self.input_dim = input_dim
    self.encoding_dim = encoding_dim

```

```

    # Encoder weights

```

```

    self.W_encoder = np.random.randn(input_dim, encoding_dim) * 0.01
    self.b_encoder = np.zeros((1, encoding_dim))

```

```

    # Decoder weights

```

```

    self.W_decoder = np.random.randn(encoding_dim, input_dim) * 0.01
    self.b_decoder = np.zeros((1, input_dim))

```

```

    self.trained = False

```

```

def _sigmoid(self, x: np.ndarray) -> np.ndarray:

```

```

    """Sigmoid activation"""
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

```

```

def encode(self, X: np.ndarray) -> np.ndarray:

```

```

    """Encode input"""
    return self._sigmoid(np.dot(X, self.W_encoder) + self.b_encoder)

```

```

def decode(self, encoded: np.ndarray) -> np.ndarray:

```

```

    """Decode encoding"""
    return self._sigmoid(np.dot(encoded, self.W_decoder) + self.b_decoder)

```

```

def train(self, X: np.ndarray, epochs: int = 50, learning_rate: float = 0.01):
    """Train autoencoder"""
    for epoch in range(epochs):
        # Forward pass
        encoded = self.encode(X)
        decoded = self.decode(encoded)

        # Compute reconstruction error
        error = X - decoded

        # Backpropagation (simplified)
        d_decoder = -error * decoded * (1 - decoded)
        d_encoder = np.dot(d_decoder, self.W_decoder.T) * encoded * (1 - encoded)

        # Update weights
        self.W_decoder -= learning_rate * np.dot(encoded.T, d_decoder) / len(X)
        self.b_decoder -= learning_rate * np.mean(d_decoder, axis=0, keepdims=True)

        self.W_encoder -= learning_rate * np.dot(X.T, d_encoder) / len(X)
        self.b_encoder -= learning_rate * np.mean(d_encoder, axis=0, keepdims=True)

    self.trained = True

```

```

def reconstruction_error(self, x: np.ndarray) -> float:
    """Calculate reconstruction error"""
    if not self.trained:
        return 0.5

    if x.ndim == 1:
        x = x.reshape(1, -1)

    encoded = self.encode(x)
    decoded = self.decode(encoded)

    error = np.mean((x - decoded) ** 2)
    return float(error)

```

```

class OneClassSVM:
    """Simplified One-Class SVM"""

    def __init__(self, nu: float = 0.1, kernel: str = 'rbf'):
        self.nu = nu
        self.kernel = kernel
        self.support_vectors = None
        self.trained = False

```

```
def _rbf_kernel(self, x1: np.ndarray, x2: np.ndarray, gamma: float = 0.1) -> float:
    """RBF kernel function"""
    return np.exp(-gamma * np.sum((x1 - x2) ** 2))
```

```
def fit(self, X: np.ndarray):
    """Fit One-Class SVM (simplified)"""
    # Store subset of data as support vectors
    n_support = min(100, len(X))
    indices = np.random.choice(len(X), n_support, replace=False)
    self.support_vectors = X[indices]
    self.trained = True
```

```
def decision_function(self, x: np.ndarray) -> float:
    """Calculate decision function value"""
    if not self.trained:
        return 0.5

    if x.ndim == 1:
        x = x.reshape(1, -1)

    # Calculate average kernel similarity to support vectors
    similarities = [self._rbf_kernel(x[0], sv) for sv in self.support_vectors]
    score = np.mean(similarities)

    # Convert to anomaly score (lower similarity = higher anomaly)
    return float(1 - score)
```

```
class LocalOutlierFactor:
    """Simplified Local Outlier Factor"""

    def __init__(self, n_neighbors: int = 20):
        self.n_neighbors = n_neighbors
        self.training_data = None
        self.trained = False

    def fit(self, X: np.ndarray):
        """Fit LOF model"""
        self.training_data = X
        self.trained = True

    def _euclidean_distance(self, x1: np.ndarray, x2: np.ndarray) -> float:
        """Calculate Euclidean distance"""
        return np.sqrt(np.sum((x1 - x2) ** 2))

    def score(self, x: np.ndarray) -> float:
        """Calculate LOF score"""
```

```

if not self.trained:
    return 0.5

if x.ndim == 1:
    x = x.reshape(1, -1)

# Find k-nearest neighbors
distances = [self._euclidean_distance(x[0], point) for point in self.training_data]
k_distances = sorted(distances)[:self.n_neighbors]

# Calculate local reachability density (simplified)
lrd = 1.0 / (np.mean(k_distances) + 1e-8)

# LOF score (simplified)
lof_score = min(lrd * 10, 1.0) # Normalize

return float(lof_score)

```

```

# =====
# GRADIENT BOOSTING ENSEMBLE
# =====

```

```
class GradientBoostingEnsemble:
```

```
    """
```

```
    Ensemble of gradient boosting models
```

```
    Includes XGBoost-style, LightGBM-style algorithms
```

```
    """
```

```
    def __init__(self):
```

```
        self.models = []
```

```
        self.trained = False
```

```
    def train(self, X: np.ndarray, y: np.ndarray):
```

```
        """Train gradient boosting ensemble"""
```

```
        # Create multiple boosting models with different parameters
```

```
        self.models = [
```

```
            SimpleGradientBoosting(n_estimators=100, learning_rate=0.1, max_depth=4),
```

```
            SimpleGradientBoosting(n_estimators=50, learning_rate=0.05, max_depth=6),
```

```
            SimpleGradientBoosting(n_estimators=150, learning_rate=0.15, max_depth=3),
```

```
        ]
```

```
        for model in self.models:
```

```
            model.train(X, y)
```

```
        self.trained = True
```

```
def predict(self, X: np.ndarray) -> np.ndarray:
    """Ensemble prediction"""
    if not self.trained:
        raise ValueError("Models not trained")

    predictions = [model.predict(X) for model in self.models]
    return np.mean(predictions, axis=0)
```

class SimpleGradientBoosting:

```
    """Simplified Gradient Boosting implementation"""
```

```
def __init__(self, n_estimators: int = 100, learning_rate: float = 0.1, max_depth: int = 3):
    self.n_estimators = n_estimators
    self.learning_rate = learning_rate
    self.max_depth = max_depth
    self.trees = []
    self.base_prediction = 0.5
```

```
def train(self, X: np.ndarray, y: np.ndarray):
    """Train gradient boosting model"""
    # Initialize with base prediction
    predictions = np.full(len(X), self.base_prediction)
```

```
    for i in range(self.n_estimators):
        # Calculate residuals
        residuals = y - predictions

        # Fit tree to residuals
        tree = self._build_tree(X, residuals, depth=0)
        self.trees.append(tree)

        # Update predictions
        tree_predictions = self._predict_tree(X, tree)
        predictions += self.learning_rate * tree_predictions
```

```
def _build_tree(self, X: np.ndarray, y: np.ndarray, depth: int) -> Dict:
```

```
    """Build decision tree"""
    if depth >= self.max_depth or len(X) <= 5:
        return {'type': 'leaf', 'value': np.mean(y)}
```

```
    best_split = self._find_best_split(X, y)
```

```
    if best_split is None:
        return {'type': 'leaf', 'value': np.mean(y)}
```

```
    feature, threshold = best_split
```

```
left_mask = X[:, feature] <= threshold
right_mask = ~left_mask
```

```
return {
    'type': 'node',
    'feature': feature,
    'threshold': threshold,
    'left': self._build_tree(X[left_mask], y[left_mask], depth + 1),
    'right': self._build_tree(X[right_mask], y[right_mask], depth + 1),
}
```

```
def _find_best_split(self, X: np.ndarray, y: np.ndarray) -> Tuple[int, float]:
```

```
    """Find best split point"""
```

```
    best_mse = float('inf')
```

```
    best_split = None
```

```
    n_features = X.shape[1]
```

```
    # Try random subset of features
```

```
    features_to_try = np.random.choice(n_features, min(int(np.sqrt(n_features)), n_features), replace=False)
```

```
    for feature in features_to_try:
```

```
        values = np.unique(X[:, feature])
```

```
        # Try random subset of thresholds
```

```
        thresholds = values[::max(1, len(values) // 10)]
```

```
        for threshold in thresholds:
```

```
            left_mask = X[:, feature] <= threshold
```

```
            right_mask = ~left_mask
```

```
            if np.sum(left_mask) < 2 or np.sum(right_mask) < 2:
```

```
                continue
```

```
            left_mse = np.var(y[left_mask]) * np.sum(left_mask)
```

```
            right_mse = np.var(y[right_mask]) * np.sum(right_mask)
```

```
            total_mse = left_mse + right_mse
```

```
            if total_mse < best_mse:
```

```
                best_mse = total_mse
```

```
                best_split = (feature, threshold)
```

```
    return best_split
```

```
def _predict_tree(self, X: np.ndarray, tree: Dict) -> np.ndarray:
```

```
    """Get predictions from a single tree"""
```

```
predictions = np.zeros(len(X))
```

```
for i, x in enumerate(X):  
    predictions[i] = self._predict_single(x, tree)
```

```
return predictions
```

```
def _predict_single(self, x: np.ndarray, tree: Dict) -> float:
```

```
    """Get prediction for a single sample"""
```

```
    if tree['type'] == 'leaf':  
        return tree['value']
```

```
    if x[tree['feature']] <= tree['threshold']:  
        return self._predict_single(x, tree['left'])  
    else:  
        return self._predict_single(x, tree['right'])
```

```
def predict(self, X: np.ndarray) -> np.ndarray:
```

```
    """Make predictions"""
```

```
    predictions = np.full(len(X), self.base_prediction)
```

```
    for tree in self.trees:  
        predictions += self.learning_rate * self._predict_tree(X, tree)
```

```
    # Apply sigmoid to get probabilities
```

```
    return 1 / (1 + np.exp(-predictions))
```

```
# =====  
# REINFORCEMENT LEARNING AGENT  
# =====
```

```
class ReinforcementLearningAgent:
```

```
    """
```

```
    RL Agent for optimal fraud response strategy
```

```
    Uses Deep Q-Network (DQN) approach
```

```
    """
```

```
def __init__(self, state_dim: int = 50, action_dim: int = 5):
```

```
    self.state_dim = state_dim
```

```
    self.action_dim = action_dim
```

```
    # Q-Network
```

```
    self.q_network = SimpleNeuralNetwork(  
        input_dim=state_dim,
```

```
        layers=[
```

```
            {'type': 'dense', 'units': 64, 'activation': 'relu'},
```

```

        {'type': 'dense', 'units': 32, 'activation': 'relu'},
        {'type': 'dense', 'units': action_dim, 'activation': 'sigmoid'},
    ]
)

```

Experience replay buffer

```

self.memory = []
self.max_memory = 10000

```

Exploration parameters

```

self.epsilon = 1.0
self.epsilon_decay = 0.995
self.epsilon_min = 0.01

```

Actions

```

self.actions = [
    'ALLOW',
    'MONITOR',
    'WARN',
    'BLOCK',
    'BLOCK_AND_ENGAGE_HONEYPOT'
]

```

```

def get_action(self, state: np.ndarray, training: bool = False) -> str:

```

```

    """

```

```

    Select action using epsilon-greedy policy

```

```

    """

```

```

    if training and np.random.random() < self.epsilon:

```

```

        # Explore: random action

```

```

        action_idx = np.random.randint(0, self.action_dim)

```

```

    else:

```

```

        # Exploit: best action from Q-network

```

```

        if state.ndim == 1:

```

```

            state = state.reshape(1, -1)

```

```

        q_values = self.q_network.predict(state)

```

```

        action_idx = np.argmax(q_values[0])

```

```

    return self.actions[action_idx]

```

```

def store_experience(self, state: np.ndarray, action: int, reward: float, next_state: np.ndarray, done: bool):

```

```

    """Store experience in replay buffer"""

```

```

    self.memory.append((state, action, reward, next_state, done))

```

```

    if len(self.memory) > self.max_memory:

```

```

        self.memory.pop(0)

```

```

def train_step(self, batch_size: int = 32, gamma: float = 0.95):
    """Train Q-network on batch of experiences"""
    if len(self.memory) < batch_size:
        return

    # Sample random batch
    batch_indices = np.random.choice(len(self.memory), batch_size, replace=False)
    batch = [self.memory[i] for i in batch_indices]

    states = np.array([exp[0] for exp in batch])
    actions = np.array([exp[1] for exp in batch])
    rewards = np.array([exp[2] for exp in batch])
    next_states = np.array([exp[3] for exp in batch])
    dones = np.array([exp[4] for exp in batch])

    # Get current Q-values
    current_q = self.q_network.predict(states)

    # Get next Q-values
    next_q = self.q_network.predict(next_states)

    # Update Q-values
    for i in range(batch_size):
        if dones[i]:
            target = rewards[i]
        else:
            target = rewards[i] + gamma * np.max(next_q[i])

        current_q[i][actions[i]] = target

    # Train network
    self.q_network.train(states, current_q, epochs=1)

    # Decay epsilon
    if self.epsilon > self.epsilon_min:
        self.epsilon *= self.epsilon_decay

```

```

# =====
# BAYESIAN INFERENCE LAYER
# =====

```

```

class BayesianInference:
    """
    Bayesian inference for uncertainty quantification
    """

```

```

def __init__(self):
    # Prior probabilities
    self.prior_fraud = 0.1 # 10% base rate for fraud
    self.prior_legitimate = 0.9

    # Likelihood parameters (updated based on evidence)
    self.fraud_indicators = {
        'urgency': {'fraud': 0.8, 'legitimate': 0.2},
        'credential_request': {'fraud': 0.95, 'legitimate': 0.05},
        'payment_redirect': {'fraud': 0.9, 'legitimate': 0.1},
        'authority_impersonate': {'fraud': 0.85, 'legitimate': 0.15},
    }

def calculate_posterior(self, evidence: Dict[str, bool]) -> Dict[str, float]:
    """
    Calculate posterior probability using Bayes' theorem
    
$$P(\text{Fraud}|\text{Evidence}) = P(\text{Evidence}|\text{Fraud}) * P(\text{Fraud}) / P(\text{Evidence})$$

    """
    # Calculate likelihoods
    likelihood_fraud = 1.0
    likelihood_legitimate = 1.0

    for indicator, present in evidence.items():
        if indicator in self.fraud_indicators:
            if present:
                likelihood_fraud *= self.fraud_indicators[indicator]['fraud']
                likelihood_legitimate *= self.fraud_indicators[indicator]['legitimate']
            else:
                likelihood_fraud *= (1 - self.fraud_indicators[indicator]['fraud'])
                likelihood_legitimate *= (1 - self.fraud_indicators[indicator]['legitimate'])

    # Calculate unnormalized posteriors
    posterior_fraud_unnorm = likelihood_fraud * self.prior_fraud
    posterior_legitimate_unnorm = likelihood_legitimate * self.prior_legitimate

    # Normalize
    total = posterior_fraud_unnorm + posterior_legitimate_unnorm

    if total == 0:
        return {'fraud': 0.5, 'legitimate': 0.5, 'confidence': 0.0}

    posterior_fraud = posterior_fraud_unnorm / total
    posterior_legitimate = posterior_legitimate_unnorm / total

    # Calculate confidence (entropy-based)
    entropy = -(posterior_fraud * np.log2(posterior_fraud + 1e-8) +
                posterior_legitimate * np.log2(posterior_legitimate + 1e-8))

```

```
confidence = 1 - (entropy / np.log2(2)) # Normalized entropy
```

```
return {  
    'fraud': posterior_fraud,  
    'legitimate': posterior_legitimate,  
    'confidence': confidence,  
}
```

```
# =====  
# META-LEARNING AGGREGATOR  
# =====
```

```
class MetaLearningAggregator:
```

```
    """
```

```
    Meta-learning layer that learns to combine predictions from all models
```

```
    """
```

```
def __init__(self, n_base_models: int = 10):
```

```
    self.n_base_models = n_base_models
```

```
    # Meta-learner (learns weights for base models)
```

```
    self.meta_learner = SimpleNeuralNetwork(  
        input_dim=n_base_models,
```

```
        layers=[
```

```
            {'type': 'dense', 'units': 32, 'activation': 'relu'},
```

```
            {'type': 'dense', 'units': 16, 'activation': 'relu'},
```

```
            {'type': 'dense', 'units': 1, 'activation': 'sigmoid'},
```

```
        ]
```

```
    )
```

```
    self.trained = False
```

```
def train(self, base_predictions: np.ndarray, y_true: np.ndarray):
```

```
    """
```

```
    Train meta-learner on base model predictions
```

```
    base_predictions: (n_samples, n_base_models)
```

```
    y_true: (n_samples,)
```

```
    """
```

```
    self.meta_learner.train(base_predictions, y_true.reshape(-1, 1), epochs=50)
```

```
    self.trained = True
```

```
def aggregate(self, base_predictions: np.ndarray) -> float:
```

```
    """
```

```
    Aggregate base model predictions
```

```
    base_predictions: (n_base_models,)
```

```
    """
```

```

if not self.trained:
    # If not trained, use simple averaging
    return float(np.mean(base_predictions))

if base_predictions.ndim == 1:
    base_predictions = base_predictions.reshape(1, -1)

meta_prediction = self.meta_learner.predict(base_predictions)
return float(meta_prediction[0])

```

```

# =====
# ADVERSARIAL ROBUSTNESS MODULE
# =====

```

```

class AdversarialDefense:
    """
    Defense against adversarial attacks on fraud detection
    """

    def __init__(self):
        self.noise_levels = [0.01, 0.05, 0.1] # Different noise levels for robustness testing

    def detect_adversarial_input(self, features: np.ndarray, model_predictions: List[float]) -> Dict[str, Any]:
        """
        Detect if input might be adversarially crafted
        """
        # Check prediction variance with noise
        noisy_predictions = []

        for noise_level in self.noise_levels:
            noise = np.random.normal(0, noise_level, features.shape)
            noisy_features = features + noise

            # Store noisy features for checking
            noisy_predictions.append(noisy_features)

        # Calculate variance in predictions
        variance = np.var(model_predictions)

        # High variance might indicate adversarial input
        is_adversarial = variance > 0.1

        return {
            'is_adversarial': is_adversarial,
            'prediction_variance': float(variance),
            'confidence': 1.0 - min(variance * 10, 1.0),

```

```
}
```

```
def add_defensive_noise(self, features: np.ndarray, noise_level: float = 0.01) -> np.ndarray:
    """
    Add defensive noise to features
    """
    noise = np.random.normal(0, noise_level, features.shape)
    return features + noise
```

```
# =====
# MASTER FRAUD DETECTION SYSTEM
# =====
```

```
class MasterFraudDetectionSystem:
```

```
    """
    Ultimate AI/ML-based fraud detection system
    Integrates all models and provides final decision
    """
```

```
    def __init__(self):
        # Initialize all components
        self.feature_extractor = AdvancedFeatureExtractor()
        self.transformer_detector = TransformerFraudDetector()
        self.lstm_analyzer = LSTMSequenceAnalyzer()
        self.cnn_recognizer = CNNPatternRecognizer()
        self.gnn = GraphNeuralNetwork()
        self.anomaly_ensemble = AnomalyDetectionEnsemble()
        self.gradient_boosting = GradientBoostingEnsemble()
        self.dnn_ensemble = DeepNeuralNetworkEnsemble()
        self.rl_agent = ReinforcementLearningAgent()
        self.bayesian = BayesianInference()
        self.meta_learner = MetaLearningAggregator()
        self.adversarial_defense = AdversarialDefense()
```

```
        self.trained = False
```

```
        # Thresholds
```

```
        self.CRITICAL_THRESHOLD = 0.85
        self.HIGH_THRESHOLD = 0.70
        self.MEDIUM_THRESHOLD = 0.50
        self.LOW_THRESHOLD = 0.30
```

```
def train_system(self, training_data: List[Dict], labels: List[int]):
    """
    Train all models in the system
    training_data: List of message dictionaries
```

```

labels: List of 0 (legitimate) or 1 (fraud)
"""

print("Training Master Fraud Detection System...")

# Extract features from all training samples
all_features = []
for data in training_data:
    features = self.feature_extractor.extract_features(
        data['message'],
        data.get('metadata', {})
    )
    feature_vector = self._features_to_vector(features)
    all_features.append(feature_vector)

X = np.array(all_features)
y = np.array(labels)

# Train anomaly detection
print("Training anomaly detection ensemble...")
self.anomaly_ensemble.train(X)

# Train gradient boosting
print("Training gradient boosting ensemble...")
self.gradient_boosting.train(X, y)

# Train deep neural networks
print("Training deep neural network ensemble...")
self.dnn_ensemble.train(X, y)

# Get base model predictions for meta-learning
print("Training meta-learner...")
base_predictions = self._get_base_predictions(X)
self.meta_learner.train(base_predictions, y)

self.trained = True
print("Training complete!")

def _features_to_vector(self, features: Dict[str, float], target_dim: int = 50) -> np.ndarray:
    """Convert feature dictionary to fixed-size vector"""
    # Get all feature values
    values = list(features.values())

    # Pad or truncate to target dimension
    if len(values) < target_dim:
        values.extend([0.0] * (target_dim - len(values)))
    else:
        values = values[:target_dim]

```

```

return np.array(values)

def _get_base_predictions(self, X: np.ndarray) -> np.ndarray:
    """Get predictions from all base models"""
    predictions = []

    # Gradient boosting predictions
    gb_pred = self.gradient_boosting.predict(X)
    predictions.append(gb_pred)

    # DNN ensemble predictions
    dnn_pred = self.dnn_ensemble.predict(X)
    predictions.append(dnn_pred)

    # Anomaly detection (simplified)
    anomaly_scores = np.array([
        self.anomaly_ensemble.isolation_forest.anomaly_score(x)
        for x in X
    ])
    predictions.append(anomaly_scores)

    # Stack predictions
    return np.column_stack(predictions)

def analyze_message(self, message: str, metadata: Dict = None) -> FraudScore:
    """
    Comprehensive fraud analysis of a message
    Returns detailed fraud score with explanation
    """
    if metadata is None:
        metadata = {}

    # ===== LAYER 1: CRITICAL PATTERN CHECK =====
    critical_result = self._check_critical_patterns(message)
    if critical_result['is_critical']:
        return FraudScore(
            total_score=1.0,
            confidence=0.99,
            model_scores={'critical_pattern': 1.0},
            risk_level=RiskLevel.CRITICAL.value,
            explanation=[f"Critical pattern detected: {critical_result['pattern_type']}"],
            features={'critical_match': critical_result['pattern_type']}
        )

    # ===== LAYER 2: FEATURE EXTRACTION =====
    features = self.feature_extractor.extract_features(message, metadata)

```

```

feature_vector = self._features_to_vector(features)

# ===== LAYER 3: MULTI-MODEL ANALYSIS =====
model_scores = {}

# Transformer-based analysis
transformer_result = self.transformer_detector.analyze_text(message)
model_scores['transformer'] = transformer_result['fraud_probability']

# LSTM sequence analysis
lstm_result = self.lstm_analyzer.analyze_sequence(message)
model_scores['lstm'] = lstm_result['sequence_fraud_score']

# CNN pattern recognition
cnn_result = self.cnn_recognizer.analyze_patterns(message)
model_scores['cnn'] = cnn_result['cnn_fraud_score']

# Graph neural network
gnn_result = self.gnn.detect_fraud_network(message, metadata)
model_scores['gnn'] = gnn_result['network_fraud_score']

# Anomaly detection
if self.trained:
    anomaly_result = self.anomaly_ensemble.detect_anomaly(feature_vector)
    model_scores['anomaly'] = anomaly_result['ensemble_anomaly_score']

# Gradient boosting
gb_pred = self.gradient_boosting.predict(feature_vector.reshape(1, -1))
model_scores['gradient_boosting'] = float(gb_pred[0])

# Deep neural networks
dnn_pred = self.dnn_ensemble.predict(feature_vector.reshape(1, -1))
model_scores['deep_nn'] = float(dnn_pred[0])
else:
    # Use pattern-based scoring if not trained
    model_scores['anomaly'] = features.get('credential_harvest_match_score', 0) / 100
    model_scores['gradient_boosting'] = features.get('payment_redirect_match_score', 0) / 100
    model_scores['deep_nn'] = features.get('urgency_tactics_match_score', 0) / 100

# ===== LAYER 4: BAYESIAN INFERENCE =====
evidence = {
    'urgency': features.get('urgency_tactics_match_count', 0) > 0,
    'credential_request': features.get('credential_harvest_match_count', 0) > 0,
    'payment_redirect': features.get('payment_redirect_match_count', 0) > 0,
    'authority_impersonate': features.get('authority_impersonate_match_count', 0) > 0,
}

```

```

bayesian_result = self.bayesian.calculate_posterior(evidence)
model_scores['bayesian'] = bayesian_result['fraud']

# ===== LAYER 5: META-LEARNING AGGREGATION =====
if self.trained and len(model_scores) >= 3:
    base_predictions = np.array(list(model_scores.values()))[:10])

    # Pad if necessary
    if len(base_predictions) < 10:
        base_predictions = np.pad(base_predictions, (0, 10 - len(base_predictions)))

    final_score = self.meta_learner.aggregate(base_predictions)
else:
    # Weighted average of model scores
    weights = {
        'transformer': 0.15,
        'lstm': 0.10,
        'cnn': 0.10,
        'gnn': 0.10,
        'anomaly': 0.15,
        'gradient_boosting': 0.15,
        'deep_nn': 0.15,
        'bayesian': 0.10,
    }

    final_score = sum(model_scores.get(model, 0) * weight
                      for model, weight in weights.items())

# ===== LAYER 6: ADVERSARIAL DEFENSE =====
adversarial_check = self.adversarial_defense.detect_adversarial_input(
    feature_vector,
    list(model_scores.values())
)

if adversarial_check['is_adversarial']:
    # Increase fraud score if adversarial attack detected
    final_score = min(final_score * 1.2, 1.0)

# ===== LAYER 7: CONFIDENCE CALIBRATION =====
confidence = bayesian_result['confidence'] * (1 - adversarial_check['prediction_variance'])

# ===== LAYER 8: RISK LEVEL DETERMINATION =====
if final_score >= self.CRITICAL_THRESHOLD:
    risk_level = RiskLevel.CRITICAL
elif final_score >= self.HIGH_THRESHOLD:
    risk_level = RiskLevel.HIGH
elif final_score >= self.MEDIUM_THRESHOLD:

```

```

        risk_level = RiskLevel.MEDIUM
    elif final_score >= self.LOW_THRESHOLD:
        risk_level = RiskLevel.LOW
    else:
        risk_level = RiskLevel.SAFE

# ===== GENERATE EXPLANATION =====
explanation = self._generate_explanation(features, model_scores, final_score)

return FraudScore(
    total_score=final_score,
    confidence=confidence,
    model_scores=model_scores,
    risk_level=risk_level.value,
    explanation=explanation,
    features=features
)

def _check_critical_patterns(self, message: str) -> Dict[str, Any]:
    """Check for critical fraud patterns requiring immediate block"""
    critical_patterns = {
        'credential_harvest': [
            r'(?i)(enter|provide|share|send|verify).*?(password|otp|pin|cvv)',
        ],
        'payment_redirect': [
            r'(?i)(transfer|send).*?(money|fund).*?account.*?\d{9,}',
        ],
        'authority_impersonate': [
            r'(?i)(police|fbi|irs|government).*?(arrest|warrant|legal.*?action)',
        ],
    }

    for pattern_type, patterns in critical_patterns.items():
        for pattern in patterns:
            if re.search(pattern, message):
                return {
                    'is_critical': True,
                    'pattern_type': pattern_type,
                }

    return {'is_critical': False}

def _generate_explanation(self, features: Dict, model_scores: Dict, final_score: float) -> List[str]:
    """Generate human-readable explanation"""
    explanation = []

    # Check pattern matches

```

```

if features.get('credential_harvest_match_count', 0) > 0:
    explanation.append("Message requests sensitive credentials (password, OTP, CVV)")

if features.get('urgency_tactics_match_count', 0) > 0:
    explanation.append("Uses urgency tactics to pressure immediate action")

if features.get('payment_redirect_match_count', 0) > 0:
    explanation.append("Attempts to redirect payment or money transfer")

if features.get('authority_impersonate_match_count', 0) > 0:
    explanation.append("Impersonates authority figures (police, government, bank)")

if features.get('has_generic_greeting', 0) > 0:
    explanation.append("Uses generic greeting instead of personalization")

if features.get('url_count', 0) > 0:
    explanation.append(f"Contains {int(features['url_count'])} suspicious links")

# Model consensus
high_scoring_models = [model for model, score in model_scores.items() if score > 0.7]
if len(high_scoring_models) >= 3:
    explanation.append(f"Multiple AI models ({len(high_scoring_models)}) detected fraud indicators")

if not explanation:
    explanation.append("AI analysis indicates potential fraud based on subtle patterns")

return explanation

def get_optimal_action(self, fraud_score: FraudScore) -> str:
    """
    Use RL agent to determine optimal response action
    """
    # Create state from fraud score
    state = np.array([
        fraud_score.total_score,
        fraud_score.confidence,
        len(fraud_score.explanation),
        fraud_score.model_scores.get('transformer', 0),
        fraud_score.model_scores.get('bayesian', 0),
    ])

    # Pad to required dimension
    if len(state) < self.rl_agent.state_dim:
        state = np.pad(state, (0, self.rl_agent.state_dim - len(state)))

    action = self.rl_agent.get_action(state, training=False)
    return action

```

```
# =====
# COMPLETE SYSTEM DEMONSTRATION
# =====

def demonstrate_system():
    """
    Demonstrate the complete fraud detection system
    """
    print("=" * 80)
    print("ADVANCED AI/ML FRAUD DETECTION SYSTEM - DEMONSTRATION")
    print("=" * 80)
    print()

    # Initialize system
    system = MasterFraudDetectionSystem()

    # Test messages
    test_messages = [
        {
            'message': "URGENT: Your account has been suspended. Click here to verify your password and OTP immediately to",
            'type': 'Critical Fraud'
        },
        {
            'message': "Dear Customer, we noticed unusual activity. Please transfer ₹50,000 to account 1234567890 within 2 hour",
            'type': 'Payment Fraud'
        },
        {
            'message': "Congratulations! You won ₹10 lakhs. Share your bank details and CVV to claim your prize now!",
            'type': 'Prize Scam'
        },
        {
            'message': "This is FBI. Arrest warrant issued against you for tax evasion. Pay penalty of $5000 to avoid jail.",
            'type': 'Authority Impersonation'
        },
        {
            'message': "Hi, this is a reminder about your appointment tomorrow at 3 PM. Please confirm.",
            'type': 'Legitimate'
        },
    ]

    print("\nAnalyzing test messages...\n")

    for i, test_msg in enumerate(test_messages, 1):
        print(f"\n{'='*80}")
        print(f"TEST CASE {i}: {test_msg['type']}")
```

```
print(f'{'='*80}')
print(f'Message: {test_msg['message'][:100]}...')
print()

# Analyze message
result = system.analyze_message(test_msg['message'])
```

```
# Display results
print(f'FRAUD SCORE: {result.total_score:.3f}')
print(f'CONFIDENCE: {result.confidence:.3f}')
print(f'RISK LEVEL: {result.risk_level}')
print()
```

```
print("MODEL SCORES:")
for model, score in sorted(result.model_scores.items(), key=lambda x: x[1], reverse=True)[:5]:
    print(f'  {model:20s}: {score:.3f}')
print()
```

```
print("EXPLANATION:")
for exp in result.explanation[:3]:
    print(f'  • {exp}')
print()
```

```
# Get optimal action
action = system.get_optimal_action(result)
print(f'RECOMMENDED ACTION: {action}')
print()
```

```
print("\n" + '='*80)
print("DEMONSTRATION COMPLETE")
print('='*80)
```

```
if __name__ == "__main__":
    demonstrate_system()
```